



THE BLUEPRINT

Build Your Own Football Prediction Model

The exact, honest playbook behind the prediction engine you've been watching — the data, the math, the models, and the traps to avoid. No black boxes. No hype.

BEGINNER FRIENDLY · PYTHON · 100% FREE TOOLS

It's **statistics**, not magic

The single most important thing to understand before you build anything.

Football scores aren't random — but they're not fully predictable either. They follow a pattern statisticians have known about for decades: the **Poisson distribution**, which describes how often rare, independent events (like goals) happen in a fixed window (like 90 minutes).

That single insight is the whole foundation. If you can estimate **how many goals each team is likely to score** against a specific opponent, the Poisson distribution hands you the probability of every possible scoreline — and from there, the chance each team wins or draws.

WHY THIS BEATS "AI" FOR BEGINNERS

A Poisson model needs very little data, runs in seconds, and — crucially — you can **explain every prediction**. When your audience asks "why does it favor Morocco?", you have a real answer. Black-box AI can't do that.

The three numbers that drive everything

A

Attack rating

How many goals a team tends to score, adjusted for how good the opponents were.

D

Defense rating

How many goals a team tends to concede, again opponent-adjusted.

H

Home / location factor

A small boost for hosts or home teams — real and measurable in the data.

Expected goals = league average × your attack × their defensive weakness × location.

Where to get good data (free)

Your model is only as good as what you feed it. Start here.

Source	What you get	Best for
martj42 / international_results (GitHub)	Every men's international since 1872 (~49k matches): date, teams, score, tournament	International / World Cup models. The go-to.
football-data.co.uk	Club league results + betting odds, many seasons, free CSVs	League models (Premier League, etc.)
Kaggle football datasets	Curated CSVs, often pre-cleaned, with discussion	Quick starts, learning
FBref / StatsBomb Open Data	Shot-level data, expected goals (xG), passing, advanced stats	Advanced models that use shot quality
eloratings.net	Elo strength ratings for national teams	A ready-made strength feature
Official FIFA rankings	Ranking points per nation, updated monthly	A baseline strength signal

HOW TO PULL IT IN PYTHON

Most of these are CSV files. One line gets you started:

```
import pandas as pd
df = pd.read_csv("results.csv", parse_dates=["date"])
df = df.dropna(subset=["home_score", "away_score"]) # keep played games
```

The golden rules of data

- ▶ **Recency matters more than volume.** A team's 2019 form barely predicts 2026. Weight recent matches heavily (a "half-life" of 12–18 months works well).
- ▶ **Down-weight friendlies.** Teams experiment in friendlies; results lie. Count them at half weight or less.
- ▶ **Adjust for opponent.** Scoring 3 against Brazil $\neq$ scoring 3 against a minnow. Good ratings account for who you played.
- ▶ **More matches per team = more trust.** Shrink extreme ratings for teams with few games toward the average.

Synthetic vs. real vs. **live** data

Knowing which one you're using keeps you honest — and keeps your content credible.

Synthetic data

Numbers you **generate or hand-pick** to represent reality — e.g. assigning each team a strength based on its FIFA ranking. Fast to set up and great for prototyping or when real data is missing.

The catch: it reflects *your assumptions*, not what actually happened on the pitch. Never claim a synthetic model is "trained" — it isn't. It's calibrated by hand.

Real (historical) data

Actual past match results. When you compute team ratings **from real games**, you're now genuinely *fitting* (or "training") your model on evidence. This is the upgrade that takes a project from toy to credible.

This is where most good models live.

Live / streaming data

Results and stats that update **as the tournament unfolds**. After each match you blend the new result into your ratings, so the model keeps learning. Great for content — your audience watches it evolve.

The catch: a few games is a tiny sample. Blend gently (e.g. 80% old rating, 20% new) so one fluke result doesn't break a team's rating.

THE VOCABULARY THAT KEEPS YOU CREDIBLE

Synthetic → say "**calibrated.**" Real historical → you've earned "**trained / fitted.**" Updating after games → "**live-updating / recalibrating.**" Using the right word is the difference between sounding like an engineer and getting called out in the comments.

Different ways to predict a match

There's no single "right" model. Each trades simplicity for power. Pick for your goal.

Model	What it does	Data needed	Explains itself?
Poisson (start here)	Models goal rates; gives full scoreline + win/draw/loss probabilities	Low	Yes ✓
Dixon–Coles	Poisson + a correction for low-scoring games and draws	Low	Yes ✓
Elo ratings	A single strength number per team, updated after each game	Low	Yes ✓
Bivariate / Skellam	Models goal <i>difference</i> directly; handles score correlation	Medium	Mostly
Logistic regression	Predicts win/draw/loss from features you choose	Medium	Yes ✓
XGBoost / Random Forest	Machine learning on many features; finds complex patterns	High	Hard ✗
Neural networks	Deep learning; powerful but data-hungry and opaque	Very high	No ✗

What is XGBoost?

Imagine hundreds of tiny decision trees, each fixing the last one's mistakes, all voting together. It's the workhorse of tabular machine learning and very accurate — **if** you have thousands of matches and rich features. For international football (teams play ~10 games a year) data is often too thin, and it can't easily tell you *why* it predicted a result.

So why start with Poisson?

It hits the sweet spot: tiny data needs, instant to run, produces the full scoreline matrix you need for visuals, and **every number is explainable**. Pro bookmakers built the entire field on exactly this. Master it first; add ML later only when you have the data to justify it.

THE SMART UPGRADE PATH

Poisson → add real data → Dixon–Coles correction → Monte Carlo tournament simulation → shot-based xG features → *then* consider XGBoost. Don't skip steps; each one teaches you what the next one needs.

Your 5-step build

From empty notebook to first prediction. All free, all in Python.

1 Get the data

Download a results CSV (start with the `international_results` dataset). Load with pandas, drop unplayed games.

2 Rate every team

From recent matches, compute opponent-adjusted Attack & Defense ratings. Weight recent games more; down-weight friendlies.

3 Predict expected goals

For any matchup: $\lambda = \text{avg_goals} \times \text{AttackA} \times \text{DefenseB} \times \text{home}$. One number per team.

4 Build the scoreline matrix

Use `scipy.stats.poisson` to get every score 0–0 to 5–5, then sum into Win / Draw / Loss %.

5 Validate & visualize

Backtest on unseen games. Then chart it — heatmaps and bars make great content.

The stack

```
# everything you need — all free, runs in Google Colab
import pandas as pd          # data
import numpy as np          # math
from scipy.stats import poisson # the model
import matplotlib.pyplot as plt # visuals
```

HOW TO TEST IT HONESTLY (most people skip this)

Split your data by date: build ratings on everything *before* a cutoff, then predict matches *after* it — games the model never saw. Track two things: **outcome accuracy** (did it pick the right result?) and **Brier score** (when it says 60%, does that happen 60% of the time?). Aim for **300–500+ test matches** for a stable read. A single tournament is too noisy to "prove" anything.

The truths most accounts **won't tell you**

These keep your predictions credible — and protect you when a pick misses.

1 · A single scoreline is never likely

Even the top exact score rarely exceeds ~13% probability. **Predict winners and percentages**, not exact scores, as your headline.

2 · The model rarely picks draws

About 1 in 4 real games end level, but a draw is almost never the single *most likely* outcome. This is math, not a bug — flag "high draw risk" instead.

3 · 55% favored is not a guarantee

Own the probability, not the outcome. "The model gives them 55%" can never be proven wrong by one game. "They'll win" can.

4 · One result shouldn't swing everything

Blend new data gently. A single 5–0 fluke is mostly noise, not a permanent shift in team quality.

THE ONE-LINE PHILOSOPHY

Predict probabilities, report honestly, post the losses too. That's what separates a real model from a hype account.



Want the daily picks & the real code?

Follow along as the model predicts every match of the 2026 World Cup — wins, losses, and the full record posted after every round.

FOLLOW @YOURHANDLE